Western New England University

College of Engineering

ECE Department

EE 590: Homework 03

Question	Score
1	/ 80
Total	/ 80

Name: **Nittala Satya Surya Lakshmi Vasuki Siva Srinivas**

#Student ID – 620094, e-mail – sn620094@wne.edu

1. Design and implement a 1x8 silicon photonic power divider using multiple 1x2 multi-mode interferometers (pdk.MMI1x2Optimized1550()) from the Si-Fab PDK. You may use IPKISS Canvas tool to aid in the design/implementation of your python code. Your submission should include the following:

   (a) Power divider IPKISS Python code utilized for layout generation (.py file)

   (b) Power divider layout file (.gds file)

   (c) A summary PowerPoint/PDF displaying scattering parameter simulation results annotated with observed values for one of the ports

Design and implement a 1x8 silicon photonic power divider using multiple 1x2 multi-mode interferometers (pdk.MMI1x2Optimized1550())

By: Nittala Satya Surya Lakshmi Vasuki Siva Srinivas

#Student ID: 620094

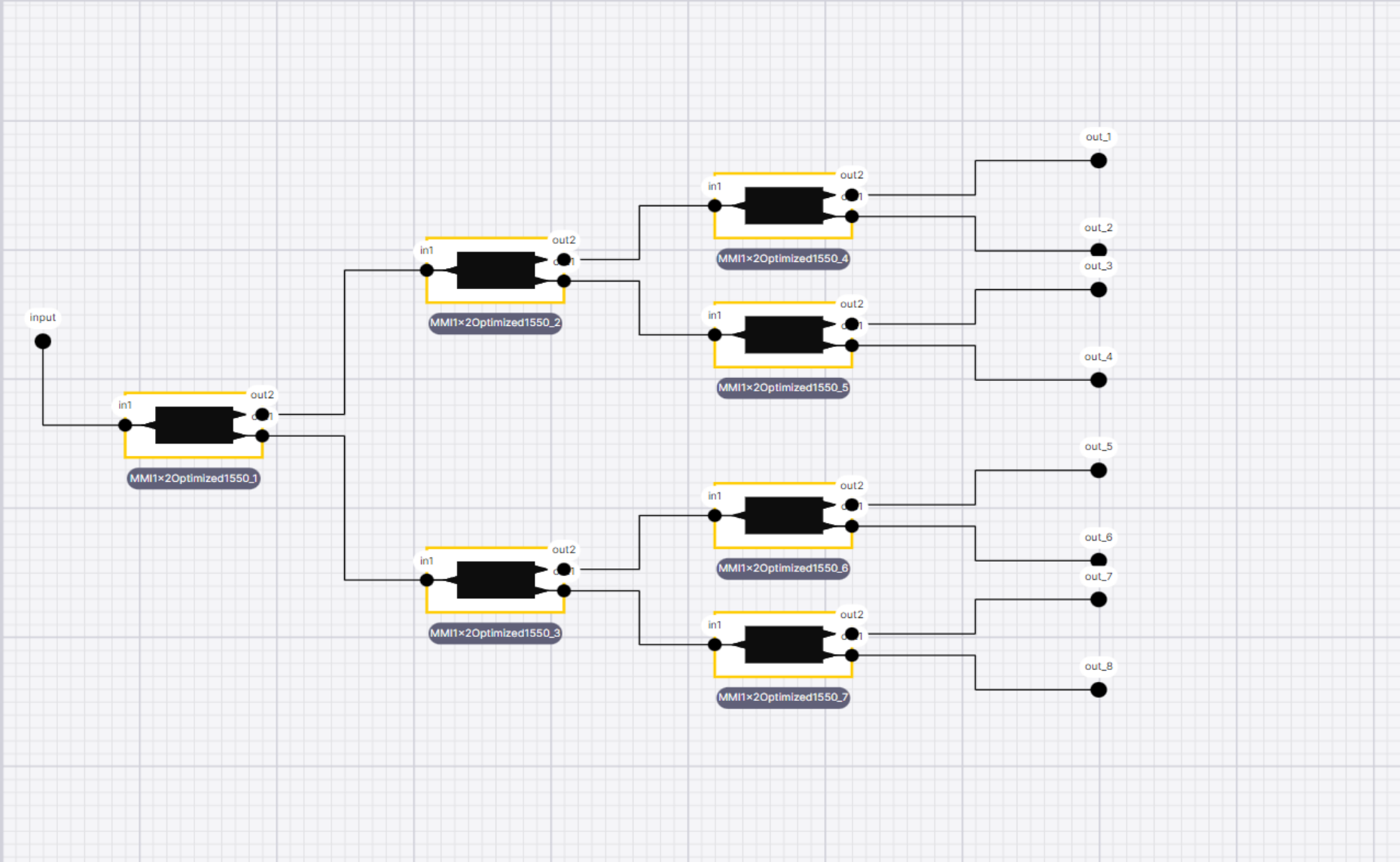
E-mail : sn620094@wne.edu

- **Design of1x8 silicon photonic power divider in IPKISS Canvas 3.10.1 .**
- **Code Template for Power Divider 1x8 in IPKISS canvas 3.10.1 .**
- **1x8 silicon photonic power divider in PYCHARM**
- **Scattering parameter simulation results in PYCHARM.**

  - 1x8 silicon photonic power divider using multiple 1x2 multi-mode interferometers.

  - Simulation results of 1x8 silicon photonic power divider.
- **Power_Divider1x8.gds[Power Divider 1x8] K7layout – [1/0]**
- **Power_Divider1x8.gds[Power Divider 1x8] K7layout – [50/2]**

Design of1x8 silicon photonic power divider in IPKISS Canvas 3.10.1



Code Template for Power Divider 1x8 in IPKISS canvas 3.10.1

Code Templates for "Power_Divider_1x8"

i3.Circuit (i3.ConnectManhattan/i3.ConnectElectrical) Template Code

```python
import si_fab.all as si_fab
import ipkiss3.all as i3

class Power_Divider_1x8(i3.Circuit):
    def _default_insts(self):
        # Variable names are auto-generated.
        # You can use the refactoring feature of your IDE to change them to more meaningful names.
        # Parameters will always be passed to the top level PCell, not its view.
        # Therefore, you should make the necessary changes and pass the parameters accordingly.
        mmi1x2optimized1550_1 = si_fab.MMI1x2Optimized1550()

        return {
            "MMI1x2Optimized1550_1": mmi1x2optimized1550_1,
            "MMI1x2Optimized1550_2": mmi1x2optimized1550_1,
            "MMI1x2Optimized1550_3": mmi1x2optimized1550_1,
            "MMI1x2Optimized1550_4": mmi1x2optimized1550_1,
            "MMI1x2Optimized1550_5": mmi1x2optimized1550_1,
            "MMI1x2Optimized1550_6": mmi1x2optimized1550_1,
            "MMI1x2Optimized1550_7": mmi1x2optimized1550_1,
        }

    def _default_specs(self):
        return [
            # To modify the specs below,
            # please refer to https://docs.lucedaphotonics.com/reference/layout/placement_routing.
            # The automatic connections save a lot of time by eliminating the need to check port names.
            # However, we can connect all these ports together, for example, using a single ConnectManhattan
            # (assuming we want the same parameters for each connection).
            # Optical connections
            i3.ConnectManhattan("MMI1x2Optimized1550_1:out2", "MMI1x2Optimized1550_2:in1"),
            i3.ConnectManhattan("MMI1x2Optimized1550_1:out1", "MMI1x2Optimized1550_3:in1"),
            i3.ConnectManhattan("MMI1x2Optimized1550_2:out2", "MMI1x2Optimized1550_4:in1"),
            i3.ConnectManhattan("MMI1x2Optimized1550_2:out1", "MMI1x2Optimized1550_5:in1"),
            i3.ConnectManhattan("MMI1x2Optimized1550_3:out2", "MMI1x2Optimized1550_6:in1"),
```

Code Templates for "Power_Divider_1x8"

i3.Circuit (i3.ConnectManhattan/i3.ConnectElectrical) Template Code

```python
            # which may not always be desirable.
            # Manual adjustments might be necessary in such cases.
            # To improve the layout, consider making the instance positions parametric and dependent
            # on other instances (using, for example, i3.PlaceRelative).
            # You may also find other minor improvements,
            # such as removing angles from instances that are set to an angle of 0.
            i3.Place("MMI1x2Optimized1550_1", (90, 340), angle=0),
            i3.Place("MMI1x2Optimized1550_2", (310, 460), angle=0),
            i3.Place("MMI1x2Optimized1550_3", (310, 220), angle=0),
            i3.Place("MMI1x2Optimized1550_4", (520, 510), angle=0),
            i3.Place("MMI1x2Optimized1550_5", (520, 410), angle=0),
            i3.Place("MMI1x2Optimized1550_6", (520, 270), angle=0),
            i3.Place("MMI1x2Optimized1550_7", (520, 170), angle=0),
        ]

    def _default_exposed_ports(self):
        return {
            "MMI1x2Optimized1550_4:out2": "out_1",
            "MMI1x2Optimized1550_4:out1": "out_2",
            "MMI1x2Optimized1550_5:out2": "out_3",
            "MMI1x2Optimized1550_5:out1": "out_4",
            "MMI1x2Optimized1550_6:out2": "out_5",
            "MMI1x2Optimized1550_6:out1": "out_6",
            "MMI1x2Optimized1550_7:out2": "out_7",
            "MMI1x2Optimized1550_7:out1": "out_8",
            "MMI1x2Optimized1550_1:in1": "input",
        }

if __name__ == "__main__":
    cell = Power_Divider_1x8(name="Power_Divider_1x8")
    cell_lo = cell.Layout()
    cell_lo.write_gdsii("Power_Divider_1x8.gds")
    cell_lo.visualize()
```

1x8 silicon photonic power divider in PYCHARM

MMI_HW_3.py Version control Current File

MMI_HW_3.py Class_example_1.py new example.py

```python
# Power_Divider_1x8 of the circuit
import ...

1 usage
class Power_Divider_1x8(i3.Circuit):
  Spacing_x = i3.PositiveNumberProperty(default=50)
  Spacing_y = i3.PositiveNumberProperty(default=25)

  def _default_insts(self):
      mmi1x2optimized1550_1 = si_fab.MMI1x2Optimized1550()

      return {
          "MMI1x2Optimized1550_1": mmi1x2optimized1550_1,
          "MMI1x2Optimized1550_2": mmi1x2optimized1550_1,
          "MMI1x2Optimized1550_3": mmi1x2optimized1550_1,
          "MMI1x2Optimized1550_4": mmi1x2optimized1550_1,
          "MMI1x2Optimized1550_5": mmi1x2optimized1550_1,
          "MMI1x2Optimized1550_6": mmi1x2optimized1550_1,
          "MMI1x2Optimized1550_7": mmi1x2optimized1550_1,
      }
```

Power_Divider_1x8 › _default_specs()

Run MMI_HW_3

```
C:\luceda\luceda_3101\python\envs\ipkiss3\python.exe "C:\Users\sn620094\OneDrive - Western New England University\luceda_photonics\EE 590\Assignment Simulations\MMI_HW_3.py"
Obtained license for ipkiss version '2023.12'.
```

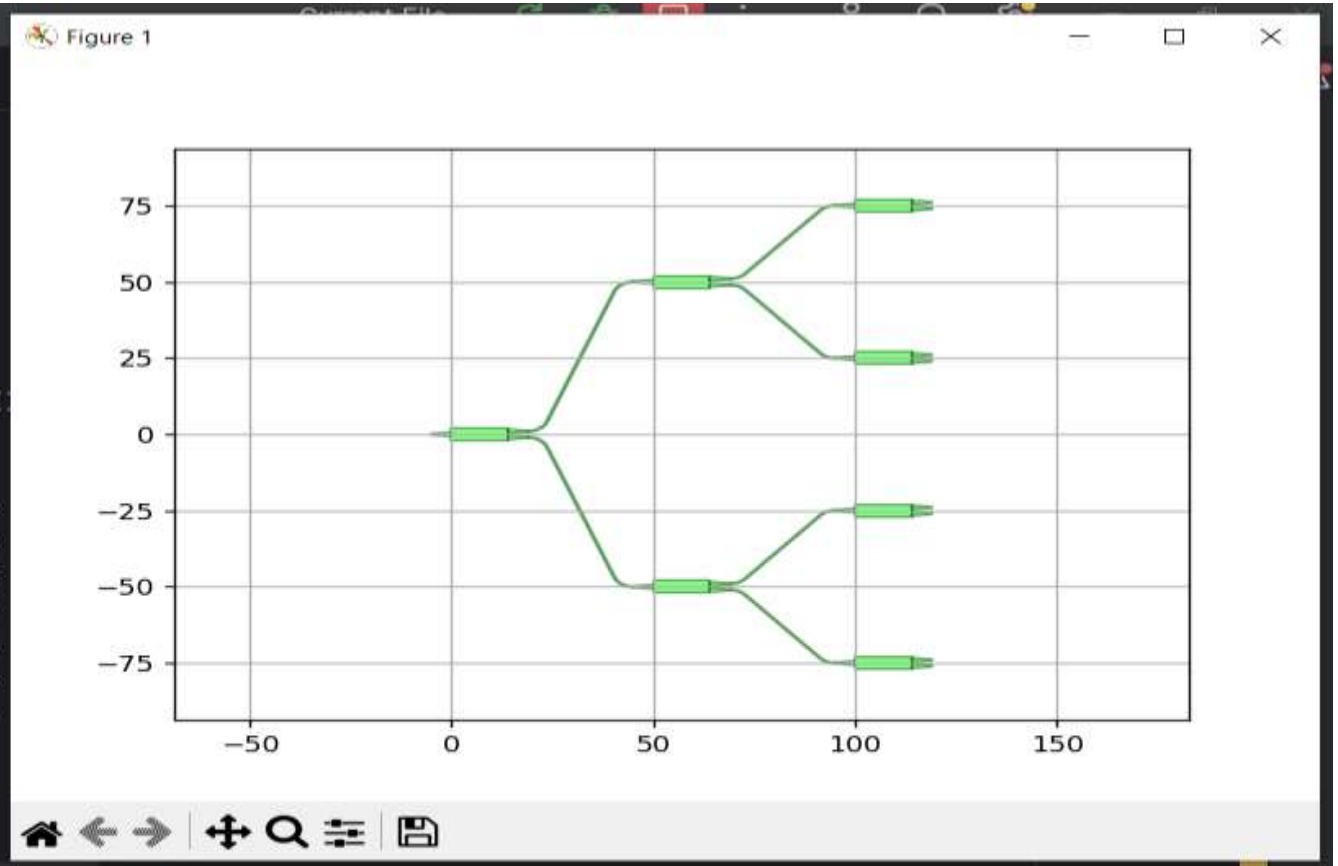
luceda_photonics › EE 590 › Assignment Simulations › MMI_HW_3.py 36:90 CRLF UTF-8 4 spaces ipkiss3

Scattering parameter simulation results in PYCHARM

MMI_HW_3.py

```python
TECH = get_technology()
1 usage
class Power_Divider_1x8(i3.Circuit):
  Spacing_x = i3.PositiveNumberProperty(default=25)
  Spacing_y = i3.PositiveNumberProperty(default=25)

  def _default_insts(self):
      mmi1x2optimized1550_1 = si_fab.MMI1x2Optimized1550(

      return {
          "MMI1x2Optimized1550_1": mmi1x2optimized1550_1,
          "MMI1x2Optimized1550_2": mmi1x2optimized1550_1,
          "MMI1x2Optimized1550_3": mmi1x2optimized1550_1,
          "MMI1x2Optimized1550_4": mmi1x2optimized1550_1,
          "MMI1x2Optimized1550_5": mmi1x2optimized1550_1,
          "MMI1x2Optimized1550_6": mmi1x2optimized1550_1,
          "MMI1x2Optimized1550_7": mmi1x2optimized1550_1,
      }

  def _default_specs(self):
      return [
          i3.Place( pos_selector: "MMI1x2Optimized1550_1",  position: (0,0)),
          i3.PlaceRelative( pos_selector1: "MMI1x2Optimized1550_2",  pos_selector2: "MMI1x2Optimized1550_1",  offset: (2*self.Spacing_x,2*self.
          i3.PlaceRelative( pos_selector1: "MMI1x2Optimized1550_3",  pos_selector2: "MMI1x2Optimized1550_1",  offset: (2*self.Spacing_x,-2*self.S
          i3.PlaceRelative( pos_selector1: "MMI1x2Optimized1550_4",  pos_selector2: "MMI1x2Optimized1550_2",  offset: (2*self.Spacing_x,self.Spac
          i3.PlaceRelative( pos_selector1: "MMI1x2Optimized1550_5",  pos_selector2: "MMI1x2Optimized1550_2",  offset: (2*self.Spacing_x,-self.Spa
          i3.PlaceRelative( pos_selector1: "MMI1x2Optimized1550_6",  pos_selector2: "MMI1x2Optimized1550_3",  offset: (2*self.Spacing_x,self.Spac
          i3.PlaceRelative( pos_selector1: "MMI1x2Optimized1550_7",  pos_selector2: "MMI1x2Optimized1550_3",  offset: (2*self.Spacing_x,-self.Spa
          i3.ConnectBend( connections: "MMI1x2Optimized1550_1:out1",  *args: "MMI1x2Optimized1550_3:in1"),
```

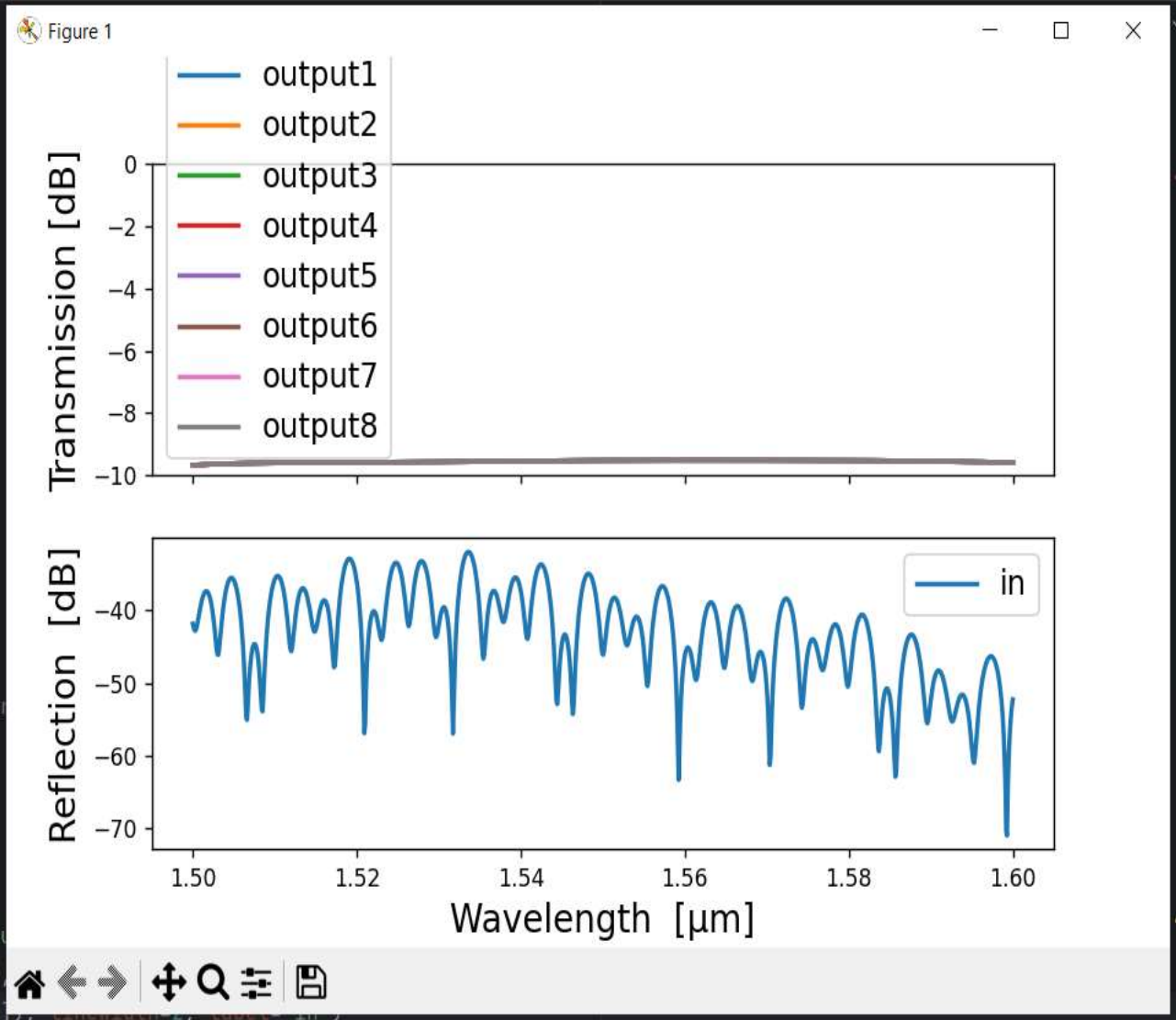
Power_Divider_1x8 › _default_specs()



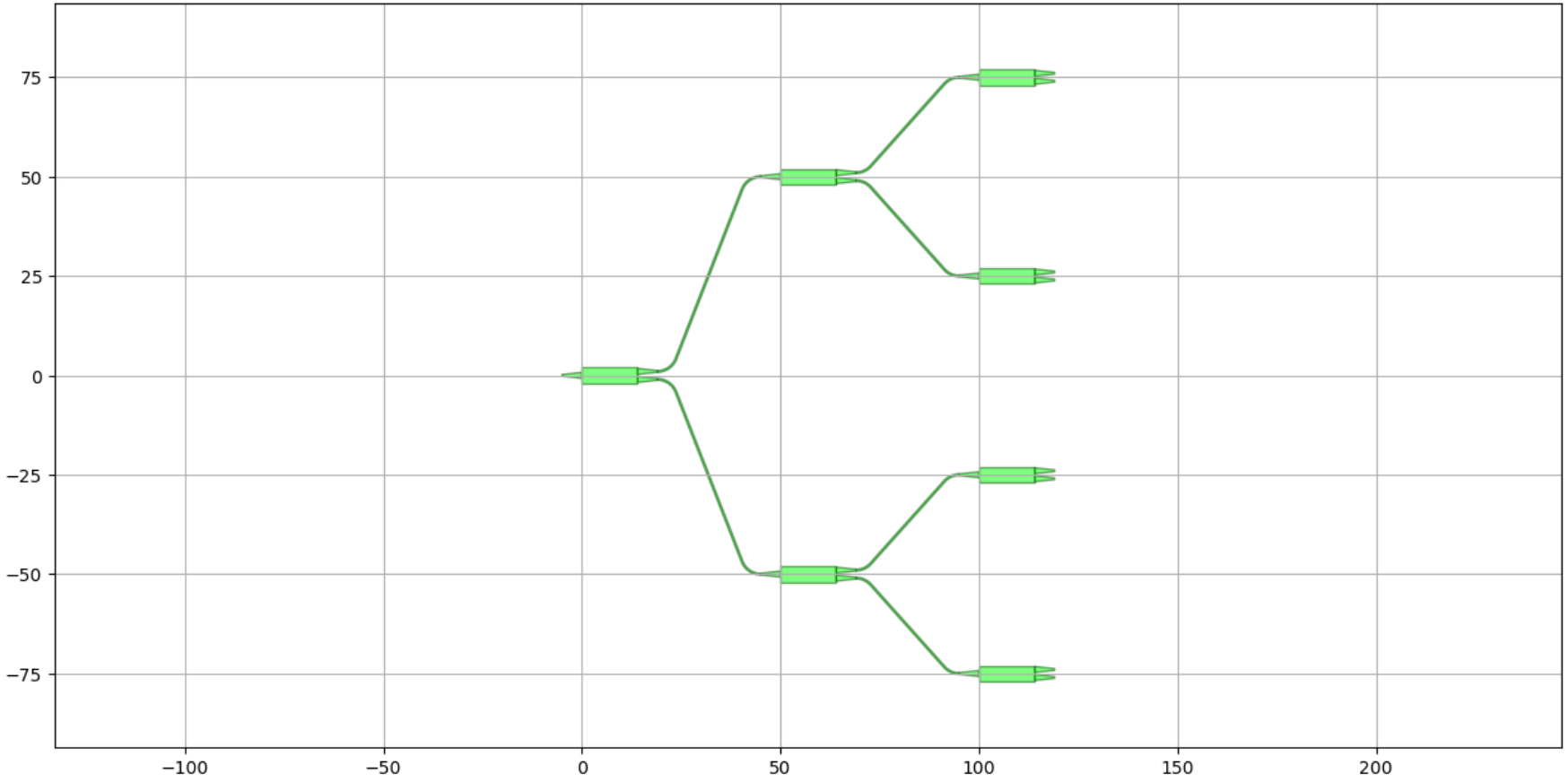
MMI_HW_3.py

```python
            "MMI1x2Optimized1550_5:out1": "output4",
            "MMI1x2Optimized1550_6:out2": "output5",
            "MMI1x2Optimized1550_6:out1": "output6",
            "MMI1x2Optimized1550_7:out2": "output7",
            "MMI1x2Optimized1550_7:out1": "output8",
        }

if __name__ == "__main__":
    cell = Power_Divider_1x8(name="Power_Divider_1x8",Spacing_y=25)
    cell_lo = cell.Layout()
    cell_lo.write_gdsii("Power_Divider_1x8.gds")
    cell_lo.visualize()

    # Generates the circuit model of our power divider
    circuit_Netlist = cell.Netlist()
    print(circuit_Netlist)
    circuit_model = cell.CircuitModel()

    # Get the S-Parameter data for each component and compute the S-Matr
    wavelengths = np.linspace( start: 1.5,  stop: 1.6,  num: 1000)
    S_total = circuit_model.get_smatrix(wavelengths=wavelengths)

    # Plot the Data
    fig, ax = plt.subplots( nrows: 2, sharex="all")
    for i in ["output1", "output2", "output3", "output4", "output5", "ou
        ax[0].plot(wavelengths, i3.signal_power_dB(S_total[i, "input"]),
    ax[1].plot(wavelengths, i3.signal_power_dB(S_total["input", "input"]), linewidth=2, label="in")
    ax[1].set_xlabel("Wavelength  [\u03BCm]", fontsize=16)  # add a label to the x-axis
    ax[0].set_ylabel("Transmission [dB]", fontsize=16)
    ax[0].set_ylim([-10, 0])
```

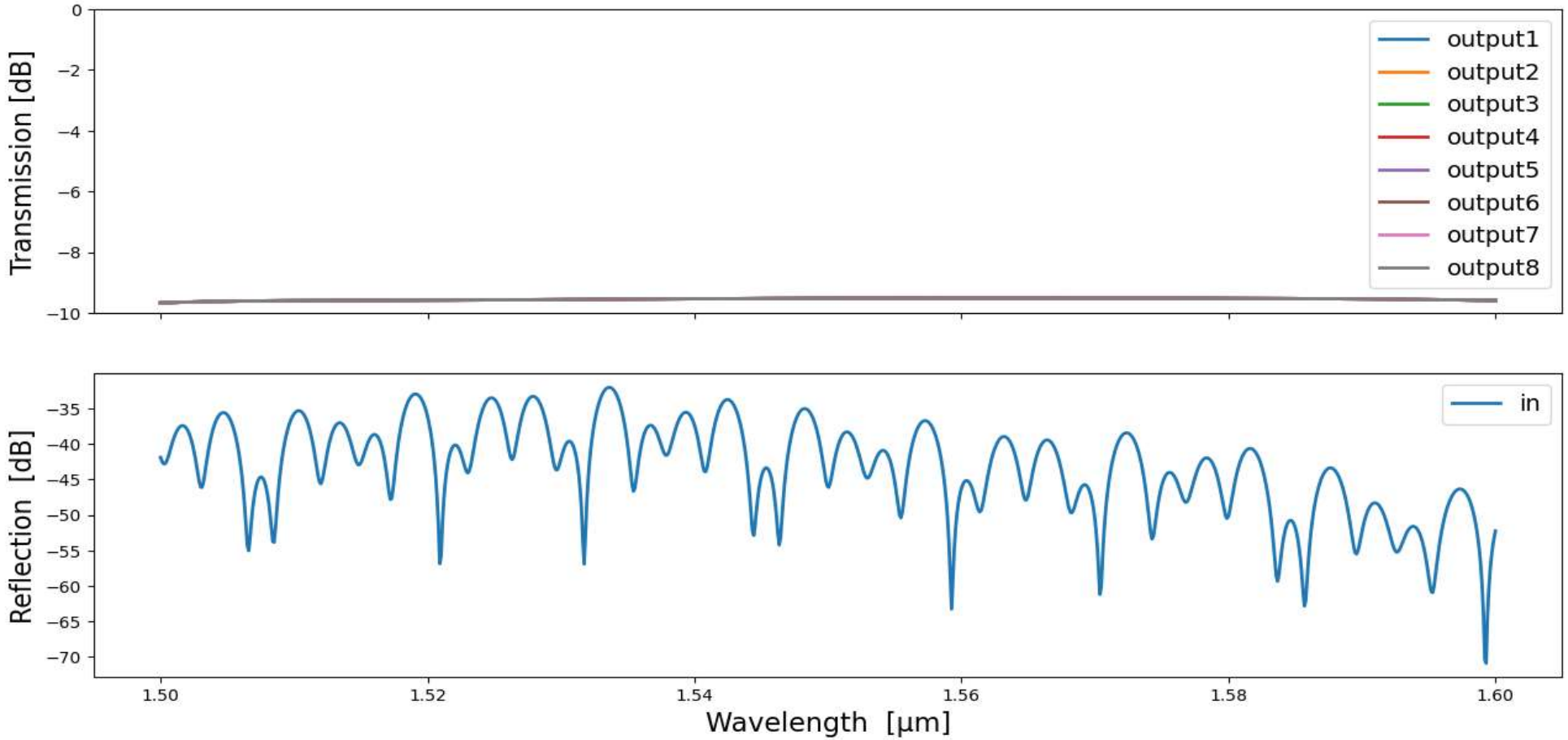
if __name__ == "__main__"



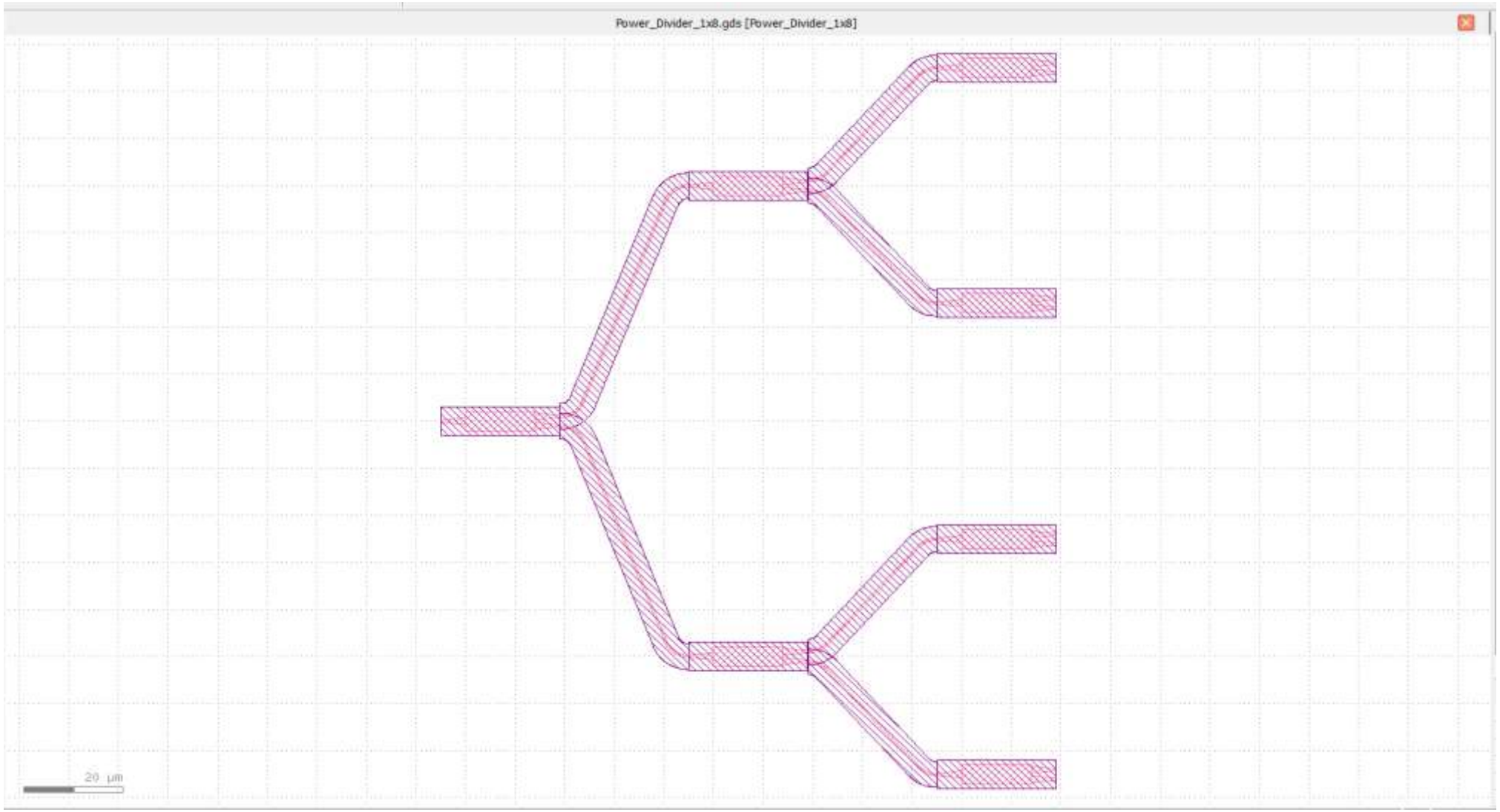
1x8 silicon photonic power divider using multiple 1x2 multi-mode interferometers



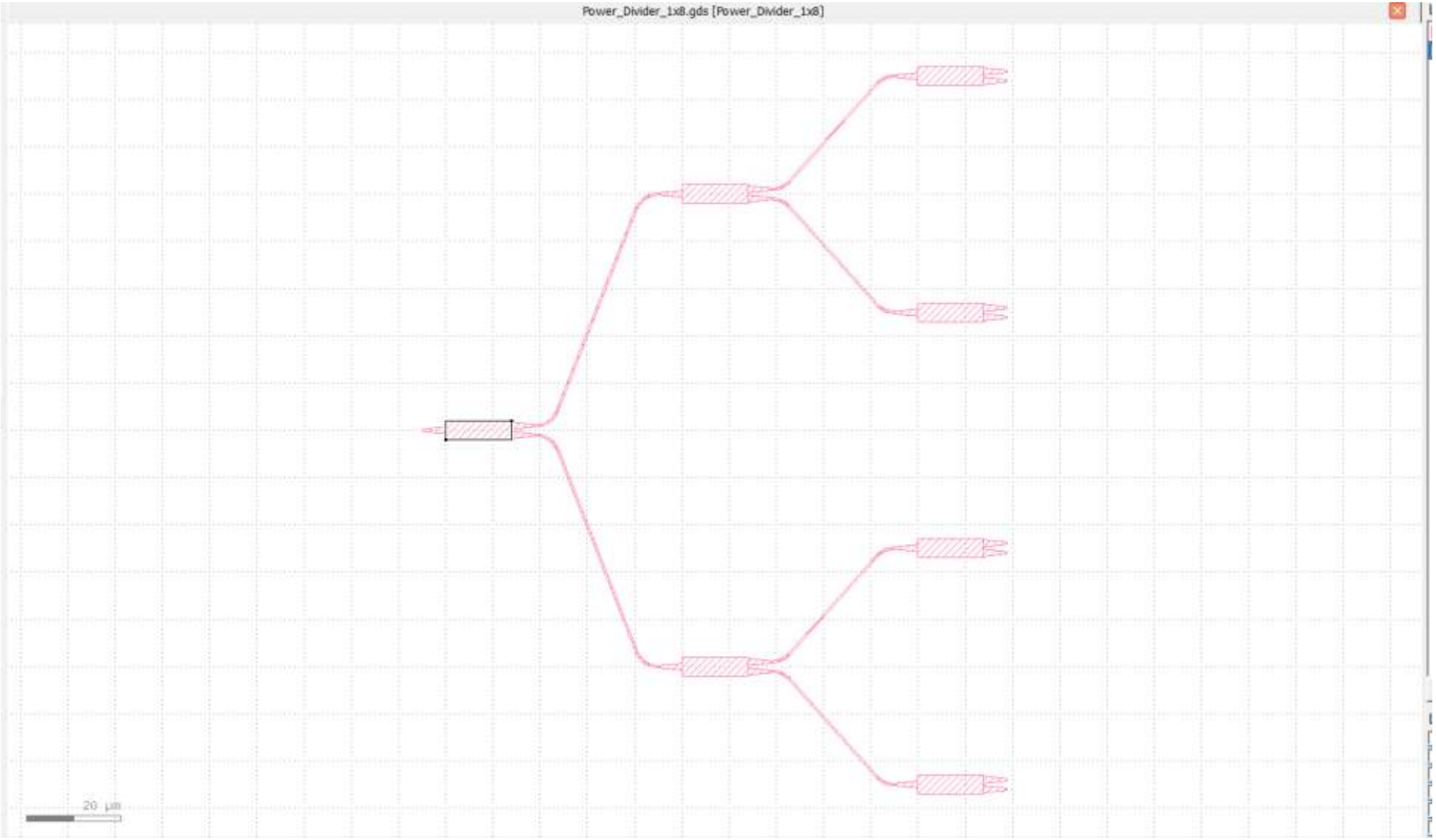
Simulation results of 1x8 silicon photonic power divider.



Power_Divider1x8.gds[Power Divider 1x8] K7layout – [1/0]



Power_Divider1x8.gds[Power Divider 1x8] K7layout – [50/2]



The End